

Report

Anqi Li, Chuyuan Xu, Jingtong He, Kaicheng Mo, Xiwei Fu

Group Members

Jingtong He(jh4977), Chuyuan Xu(cx2347), Anqi Li(al4771), Xiwei Fu(xf2290), Kaicheng Mo(km4071)

Project Title

ClimaScope: Forecasting the Empire Skies

Motivation

New York State’s climate has already warmed substantially, with increases in mean and extreme temperatures and changes in precipitation documented over the twentieth and early twenty-first centuries and projected to intensify under all plausible emissions scenarios (Lamie et al., 2024). These shifts amplify risks to human health, infrastructure and ecosystems, particularly through more frequent heat extremes, heavy rainfall and flooding, underscoring the need for decision-relevant climate information at sub-state scales (Barnes et al., 2024). While global classification schemes such as the Köppen–Geiger system provide a useful first-order view of climate regimes, they do not resolve county-level variation in multivariate weather conditions within a single state (Peel, Finlayson and McMahon, 2007).

Related Work

Using 2016–2020 daily county-level meteorological observations, this project develops an integrated, three-part statistical characterisation of New York’s near-surface climate designed to support downstream impact analyses.

First, we aggregate daily variables to the state level to describe seasonal and inter-annual variability in maximum temperature and related meteorological quantities.

Second, we construct county-specific long-term climate profiles—summaries of temperature, humidity, precipitation, solar radiation and wind speed—and apply principal component analysis (PCA) followed by k-means clustering to identify objectively defined climate types, following established multivariate climate regionalisation approaches that combine dimensionality reduction with unsupervised clustering.

Third, we treat statewide daily maximum temperature as a primary outcome and develop an ARIMAX model with Fourier seasonal terms and time-lagged meteorological covariates (humidity, vapour-pressure deficit, solar radiation and wind speed) to capture both the dominant annual cycle and shorter-term weather dependencies, building on recommended practice for ARIMA-family models with harmonic regressors in environmental time-series forecasting (Hyndman and Athanasopoulos, 2018; Bazrafshan et al., 2025).

The overarching motivation is to obtain interpretable spatial climate types, a low-dimensional representation of the main climate gradient across New York, and a calibrated forecasting model for daily heat at the state

level, thereby generating meteorological inputs that can in future be coupled to health, infrastructure or ecological outcome models in support of climate-resilient planning.

Initial Questions

Our initial questions were designed to explore New York State’s multivariate climate patterns and guide the development of a meaningful statistical characterization.

- How do seasonal and inter-annual variations in key meteorological variables manifest across New York State, and what broad climate signals can be observed at the state level? As we examined these trends, we began to consider whether certain variables contributed disproportionately to statewide variability and whether long-term warming patterns might shift these signals over time.
- To what extent do counties differ in their long-term climate profiles, and can unsupervised methods such as PCA and k-means clustering reveal distinct, objectively defined climate types? The PCA results prompted new questions about the stability of the extracted gradients, the robustness of clusters to alternative variable selections or scaling choices, and whether the identified climate types could meaningfully support downstream applications such as health or infrastructure risk modeling.
- Can a time-series forecasting model effectively capture both the dominant annual temperature cycle and shorter-term weather dependencies?

As we refined the model, additional questions arose regarding the sensitivity of forecasts to lag structures, the potential value of additional covariates, and how such forecasts might integrate with the county-level climate classifications.

Data Summary

Dataset

The datasets recording various meteorological variables were downloaded from “**Daily meteorological Gridmet variables by United States administrative boundaries**”, **Harvard Dataverse**. The dataset provides spatial aggregations of meteorological for US counties. To monitor the changing patterns in the past seasons and years, meteorological data from 2016 to 2020 on the county level were downloaded to get the closest observations available.

Key meteorological factors used for visualization and addressing our research question are:

county: Federal Information Processing Series (FIPS), the id of the counties;
date: date of the estimated meteorological variables;
sph: near-surface specific humidity (Mass fraction);
vpd: mean vapor pressure deficit (kPa);
tmmn: minimum near-surface air temperature (Kelvin);
tmmx: maximum near-surface air temperature (Kelvin);
pr: precipitation (mm, daily total);
rmin: minimum near-surface relative humidity (%);
rmax: maximum near-surface relative humidity (%);
srad: surface downwelling solar radiation (W/m^2);
vs: wind speed at 10m (m/s);
th: wind direction at 10m (degree).

Corresponding county information was downloaded from **tiger/line Shapefiles**. Due to the lapse in

federal funding, portions of the **United States Census Bureau** were unavailable. `t1_2020_state` and `t1_2024_county` were downloaded to obtain additional New York state and county information to facilitate visualizations and analysis.

Key variables in `t1_2024_county` include:

- `GEOID`: Current county identifier; a concatenation of current state FIPS code and county FIPS code;
- `STUSPS`: Current United States Postal Service (USPS) state abbreviation;
- `NAME`: Current state name;
- `NAMELSAD`: Current name and the translated LSAD code for county;
- `geometry`: the geological boundaries in the geometry of multipolygon.

Key variables in `t1_2020_state` are:

- `STATEFP`: State FIPS code;
- `STUSPS`: Current United States Postal Service (USPS) state abbreviation;
- `NAME`: Current county name.

Data Cleaning

Meteorological Factors dataset

Due to the fact that each raw dataset exceeds the maximum upload file size (>50 MB), data cleaning was performed on a local drive. The tidied dataset is available on our Github. Meteorological datasets from 2016 to 2020 were combined, and only counties in the NY states would be selected for the new dataset. The new dataset has 113274 observations, each with unique dates and FIPs code, and 6 variables with county information and meteorological factors. In later analysis, minimum and maximum near-surface air temperatures were converted from Kelvin into Celsius.

NY County Information

All the FIPs in the state of New York were first filtered out from the dataset on states, and county information was joined to serve as a supplement of county names and multipolygon geographical information. The new dataset now includes 62 unique NY counties and 6. Data points are usually stored in the two sections above and merged when needed.

Exploratory Data Analysis

Overview

This interface presents exploratory summaries of county-level meteorological conditions in New York State for the period 2016–2020. The underlying dataset links daily observations to county polygons via the FIPS county identifier (`fips`) and calendar date (`date`). Maximum near-surface air temperature (`tmmx`, recorded in Kelvin) is converted to degrees Celsius (`tmmx_C`) and used to calculate statewide daily means, the annual number of “hot” days above 30 °C, and multi-year county averages. Daily total precipitation (`pr`, in millimetres) is grouped by meteorological season to characterise the distribution and frequency of wet days, while vapor pressure deficit (`vpd`, in kPa) is analysed jointly with temperature to illustrate how atmospheric dryness changes under different heat conditions. Derived temporal variables such as `month`, `season`, and `year` organise the data in time, and geographic coordinates of county centroids (`lon`, `lat`) support the interactive map. Together, these variables provide a compact description of how temperature, moisture, and heat extremes vary across space and time in New York State.

```
meteor <- readr::read_csv("df_gen/data/meteor_2016to20_NY.csv")
```

```
ny_county <- sf::st_read("df_gen/data/NYcounty_fips/NYcounty_fips.shp",  
  quiet = TRUE  
)
```

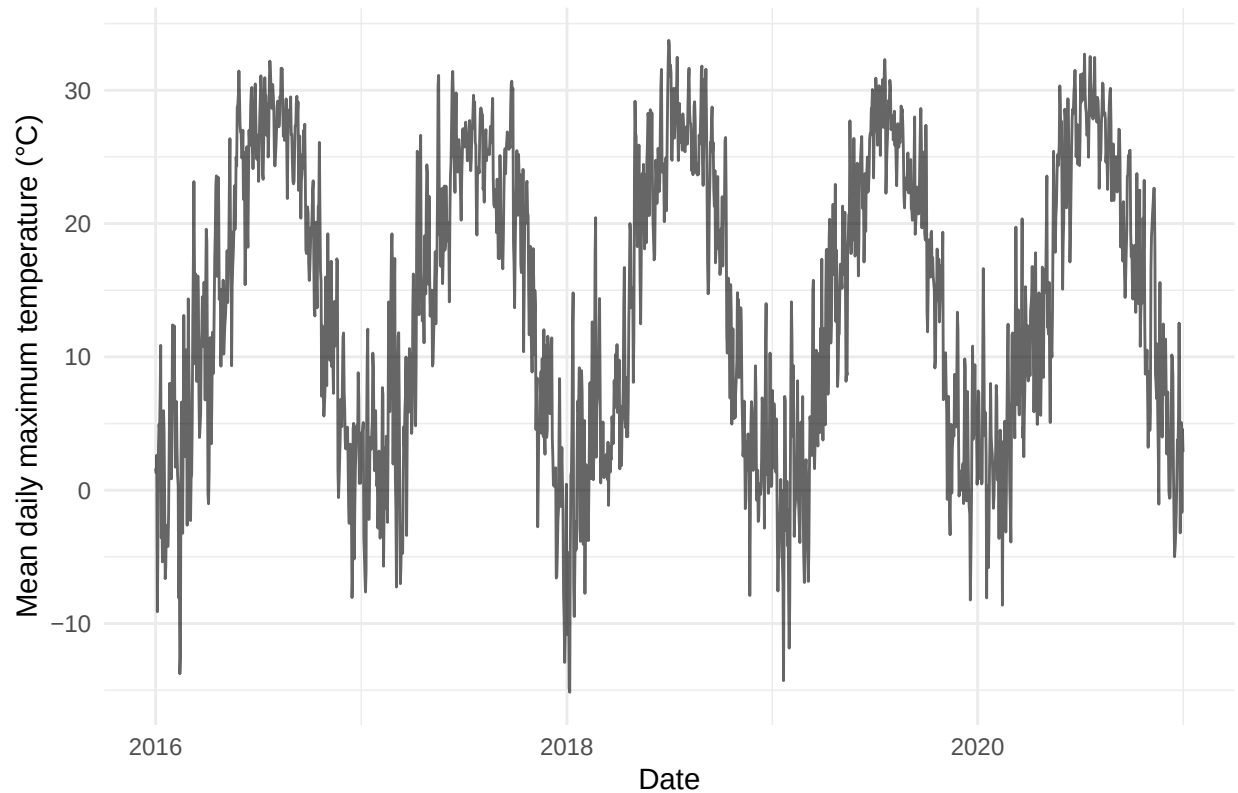
```
ny_county <- ny_county |>  
  mutate(fips = as.character(fips))  
  
meteor <- meteor |>  
  mutate(fips = as.character(fips))
```

```
ny_meteor_all <- ny_county |>  
  dplyr::left_join(meteor, by = "fips")
```

Daily statewide mean of maximum temperature in New York (2016–2020)

```
state_tmmx_ts <- ny_meteor_all |>  
  sf::st_drop_geometry() |>  
  group_by(date) |>  
  summarise(  
    tmmx_mean_K = mean(tmmx, na.rm = TRUE),  
    .groups = "drop"  
  ) |>  
  mutate(tmmx_mean_C = tmmx_mean_K - 273.15)  
  
ggplot(state_tmmx_ts, aes(x = date, y = tmmx_mean_C)) +  
  geom_line(alpha = 0.6) +  
  labs(  
    title = "Daily statewide mean of maximum temperature in New York (2016–2020)",  
    x = "Date",  
    y = "Mean daily maximum temperature (°C)"  
  ) +  
  theme_minimal()
```

Daily statewide mean of maximum temperature in New York (2016–2020)



The curve shows the mean daily maximum near-surface air temperature, averaged across all New York State counties and expressed in degrees Celsius. The pronounced peaks and troughs illustrate the strong intra-annual seasonal cycle, with warm summers and cold winters repeating over the five-year period. The series also highlights shorter-term variability and occasional extreme hot and cold days superimposed on this seasonal pattern.

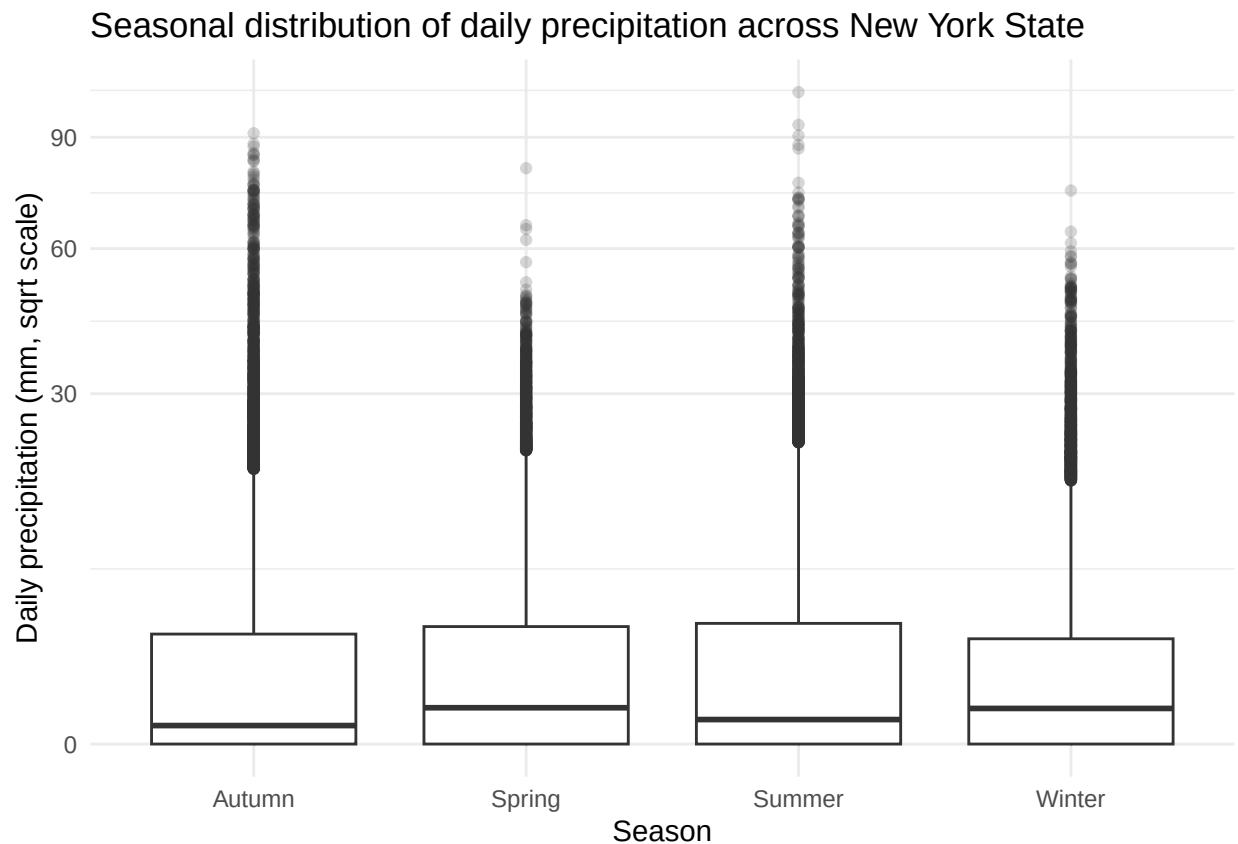
Seasonal distribution of daily precipitation across New York State

```
ny_meteor_no_geom <- ny_meteor_all |>
  sf::st_drop_geometry()

seasonal_pr <- ny_meteor_no_geom |>
  mutate(
    month = month(date),
    season = case_when(
      month %in% c(12, 1, 2) ~ "Winter",
      month %in% c(3, 4, 5) ~ "Spring",
      month %in% c(6, 7, 8) ~ "Summer",
      TRUE ~ "Autumn"
    )
  )

ggplot(seasonal_pr, aes(x = season, y = pr)) +
  geom_boxplot(outlier.alpha = 0.2) +
```

```
scale_y_sqrt(name = "Daily precipitation (mm, sqrt scale)") +
labs(
  title = "Seasonal distribution of daily precipitation across New York State",
  x = "Season"
) +
theme_minimal()
```



Box-and-whisker plots summarise the distribution of county-level daily total precipitation (mm) for each meteorological season over 2016–2020, displayed on a square-root scale to make moderate and heavy rainfall more visible. All seasons exhibit a highly right-skewed distribution, reflecting many days with little or no rain and a smaller number of intense precipitation events. Differences between seasons suggest modest shifts in both typical and extreme rainfall amounts across the annual cycle.

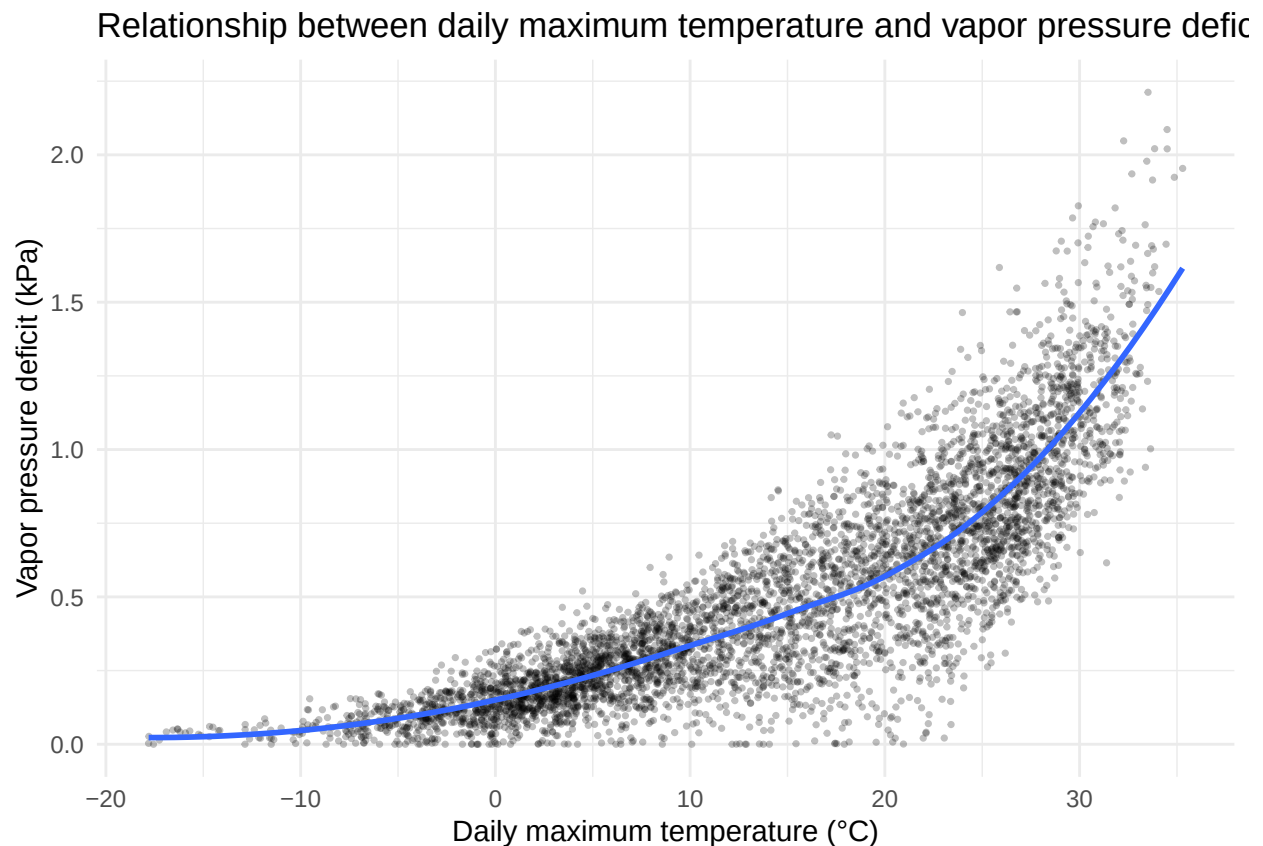
Relationship between daily maximum temperature and vapor pressure deficit

```
set.seed(123)

scatter_dat <- ny_meteor_no_geom |>
  select(fips, date, tmmx, vpd) |>
  filter(!is.na(tmmx), !is.na(vpd)) |>
  mutate(tmmx_C = tmmx - 273.15) |>
  slice_sample(n = 5000)

ggplot(scatter_dat, aes(x = tmmx_C, y = vpd)) +
```

```
geom_point(alpha = 0.25, size = 0.6) +
geom_smooth(method = "loess", se = FALSE) +
labs(
  title = "Relationship between daily maximum temperature and vapor pressure deficit",
  x = "Daily maximum temperature (°C)",
  y = "Vapor pressure deficit (kPa)"
) +
theme_minimal()
```



Each grey point represents a sampled county–day pair from 2016–2020, plotting daily maximum temperature (°C) against the corresponding vapor pressure deficit (kPa), while the blue line shows a non-parametric smooth fitted to the cloud of points. The scatter reveals a strong, non-linear positive association, with VPD increasing rapidly at higher temperatures, consistent with warmer air having a greater capacity for holding water vapour. This pattern indicates that hot conditions in New York are typically accompanied by drier atmospheric demand, which is relevant for both ecological water stress and human heat exposure.

Average number of hot days ($T_{\text{max}} \geq 30^\circ\text{C}$) per county by year

```
hot_threshold_K <- 30 + 273.15

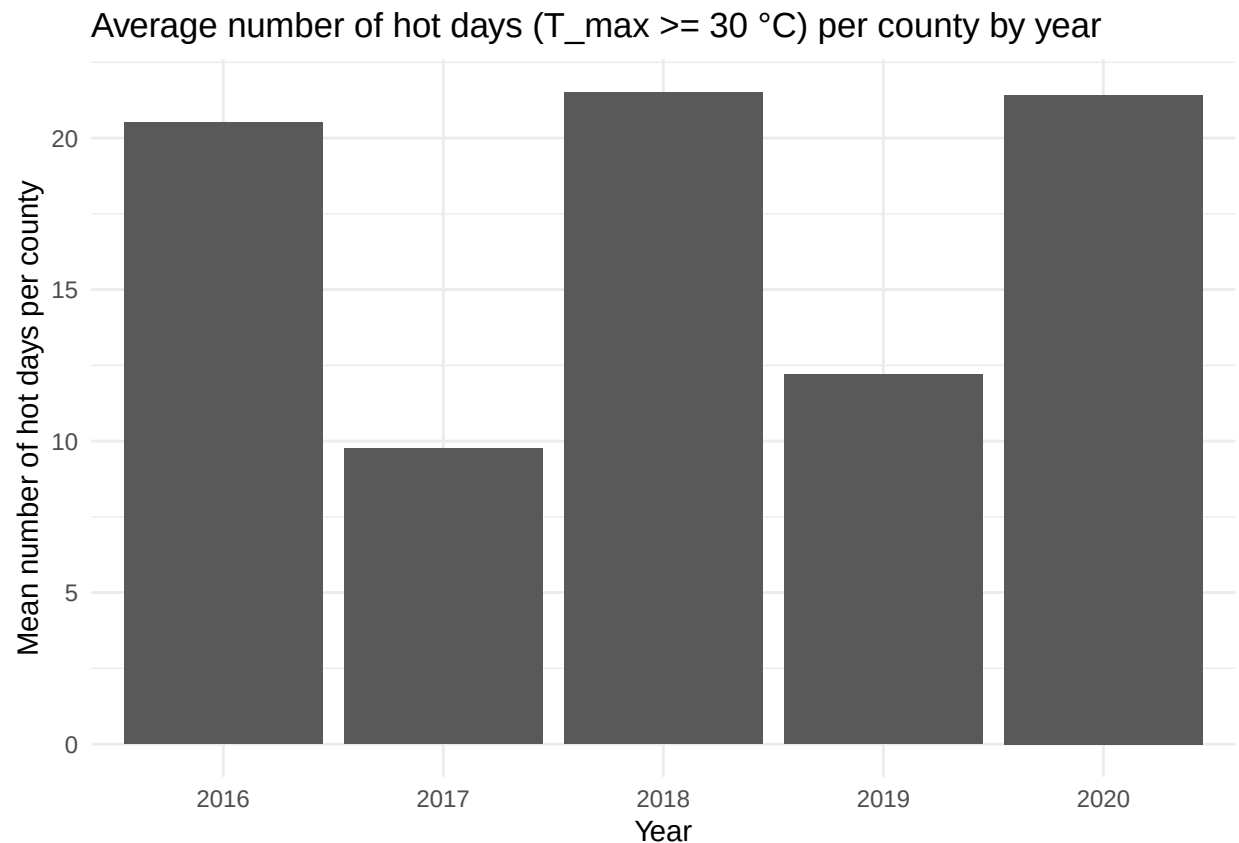
yearly_hot_days <- ny_meteor_no_geom |>
  mutate(
    year = year(date),
    is_hot = tmmx >= hot_threshold_K
```

```

) |>
group_by(fips, year) |>
summarise(
  hot_days = sum(is_hot, na.rm = TRUE),
  .groups = "drop"
) |>
group_by(year) |>
summarise(
  mean_hot_days = mean(hot_days, na.rm = TRUE),
  .groups = "drop"
)

ggplot(yearly_hot_days, aes(x = factor(year), y = mean_hot_days)) +
  geom_col() +
  labs(
    title = "Average number of hot days (T_max >= 30 °C) per county by year",
    x = "Year",
    y = "Mean number of hot days per county"
  ) +
  theme_minimal()

```



Bars display, for each calendar year from 2016 to 2020, the mean number of days per county on which the daily maximum temperature reached or exceeded 30 °C. The metric provides a simple indicator of population-level exposure to potentially hazardous heat across the state. The inter-annual differences in bar height indicate that the frequency of such hot days varies appreciably from year to year over the study period.


```

ny_meteor_tbl <- ny_meteor_all |>
  st_drop_geometry() |>
  mutate(tmmx_C = tmmx - 273.15)

county_mean_tmmx <- ny_meteor_tbl |>
  group_by(fips) |>
  summarise(
    mean_tmmx_C = mean(tmmx_C, na.rm = TRUE),
    .groups = "drop"
  )

ny_county_centroid <- ny_county |>
  dplyr::select(fips, name, geometry) |>
  st_transform(4326) |>
  mutate(centroid = st_centroid(geometry)) |>
  mutate(
    lon = st_coordinates(centroid)[, 1],
    lat = st_coordinates(centroid)[, 2]
  ) |>
  dplyr::select(-centroid)

ny_map_data <- ny_county_centroid |>
  left_join(county_mean_tmmx, by = "fips")

```

Interactive map of mean daily maximum temperature by county

```

pal <- colorNumeric(
  palette = "YlOrRd",
  domain = ny_map_data$mean_tmmx_C,
  na.color = "transparent"
)

leaflet(ny_map_data, options = leafletOptions(minZoom = 5, maxZoom = 10)) |>
  addProviderTiles("CartoDB.Positron") |>

  addPolygons(
    fillColor = ~pal(mean_tmmx_C),
    color = "#555555",
    weight = 0.5,
    fillOpacity = 0.7,
    smoothFactor = 0.2,
    highlight = highlightOptions(
      weight = 2,
      color = "#000000",
      fillOpacity = 0.9,
      bringToFront = TRUE
    ),
    label = ~paste0(
      name, " County: ",
      round(mean_tmmx_C, 1), " °C"
    ),
  ),

```

```

popup = ~paste0(
  "<strong>", name, " County</strong><br/>",
  "Mean T<sub>max</sub>: ", round(mean_tmmx_C, 1), " °C<br/>",
  "Longitude: ", round(lon, 3), "<br/>",
  "Latitude: ", round(lat, 3)
)
) |>
addCircleMarkers(
  lng = ~lon,
  lat = ~lat,
  radius = 2,
  stroke = TRUE,
  color = "#B0B0B0",
  weight = 1,
  fillColor = "#FFF7E6",
  fillOpacity = 0.5,
  popup = ~paste0(
    "<strong>", name, " County centroid</strong><br/>",
    "Mean T<sub>max</sub>: ", round(mean_tmmx_C, 1), " °C<br/>",
    "Lon: ", round(lon, 3), ", Lat: ", round(lat, 3)
  )
) |>

addLegend(
  position = "bottomright",
  pal      = pal,
  values   = ~mean_tmmx_C,
  title    = "Mean T<sub>max</sub> (°C)",
  labFormat = labelFormat(digits = 1)
)

```

County polygons are shaded according to the multi-year mean daily maximum temperature (°C), with warmer colours indicating higher long-term averages, and blue markers denoting county centroids. This interactive figure allows detailed exploration of the spatial heterogeneity in temperature across New York State and facilitates linking local climatic conditions to downstream health or environmental outcomes.

Supplement Materials

```
cat("If you would like to explore additional variables in our dataset, please click [Data Presentation]")
```

If you would like to explore additional variables in our dataset, please click Data Presentation to view more details.

K-means clustering and PCA

```

knitr::opts_chunk$set(message = FALSE, warning = FALSE)
library(dplyr)
library(lubridate)

```

```
library(sf)
library(tidyverse)
library(factoextra)
library(ggplot2)

county_fips_df <- st_read("df_gen/data/NYcounty_fips", quiet = TRUE)
weather <- read.csv("df_gen/data/meteor_2016to20_NY.csv")
```

Methodology

Data Processing and Feature Construction

```
weather2 <- weather |>
mutate(
  tmmx_c = tmmx - 273.15,
  tmmn_c = tmmn - 273.15,
  year   = year(date),
  month  = month(date)
)

county_feat <- weather2 |>
group_by(county) |>
summarise(
  mean_tmmx_c = mean(tmmx_c, na.rm = TRUE),
  mean_tmmn_c = mean(tmmn_c, na.rm = TRUE),
  mean_rmax   = mean(rmax,   na.rm = TRUE),
  mean_vpd    = mean(vpd,    na.rm = TRUE),
  mean_pr     = mean(pr,     na.rm = TRUE),
  mean_srad   = mean(srad,   na.rm = TRUE),
  mean_vs     = mean(vs,     na.rm = TRUE),
  .groups = "drop"
)
```

We summarized daily meteorological records from 2016–2020 into county-level “climate profiles.” For each county, we computed long-term averages of maximum and minimum temperature, maximum relative humidity, vapor pressure deficit, precipitation, solar radiation, and wind speed. This aggregation yields one observation per county representing its typical climate over the study period.

Standardization

```
feat_mat <- county_feat |>
select(-county) |>
scale() |>
as.matrix()
```

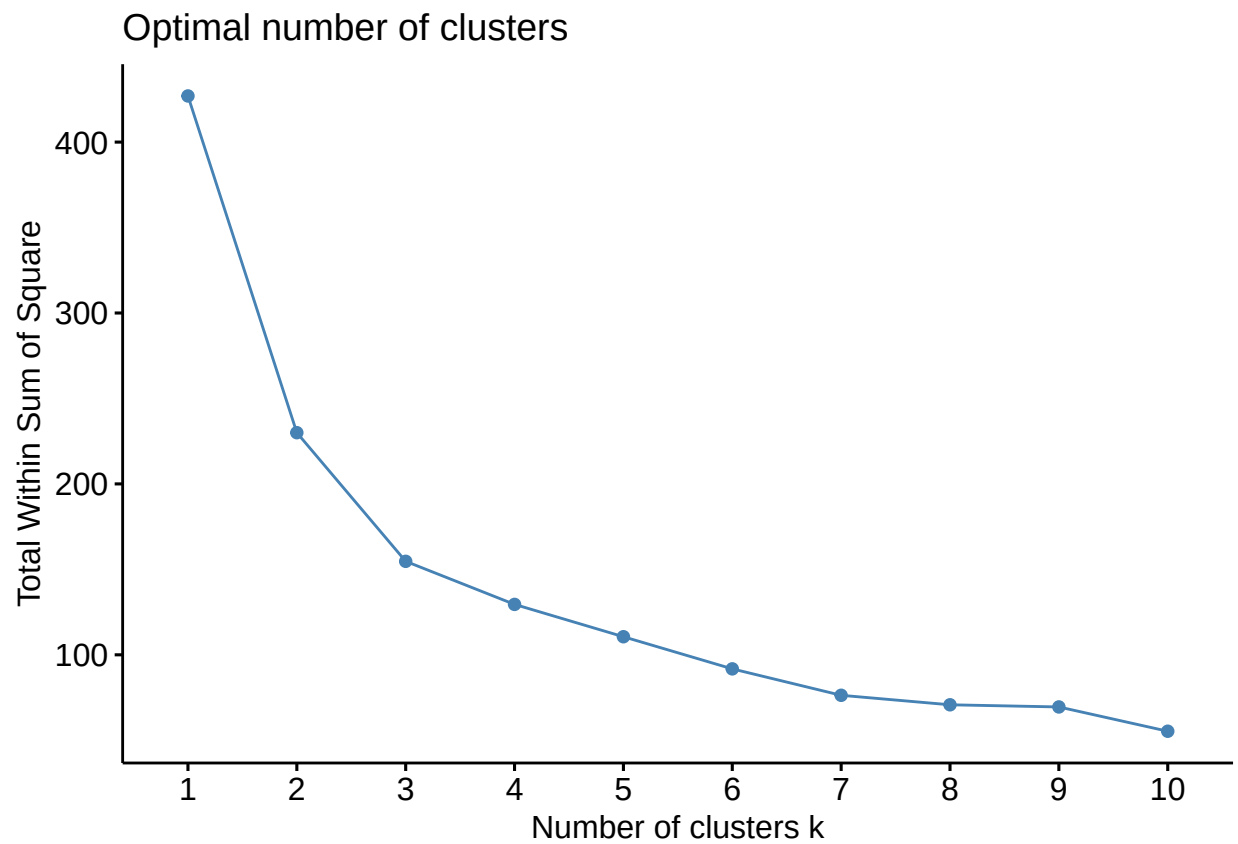
Before clustering, all continuous features were standardized to mean 0 and unit variance so that no single variable dominated the distance calculations in k-means.

Clustering Approach

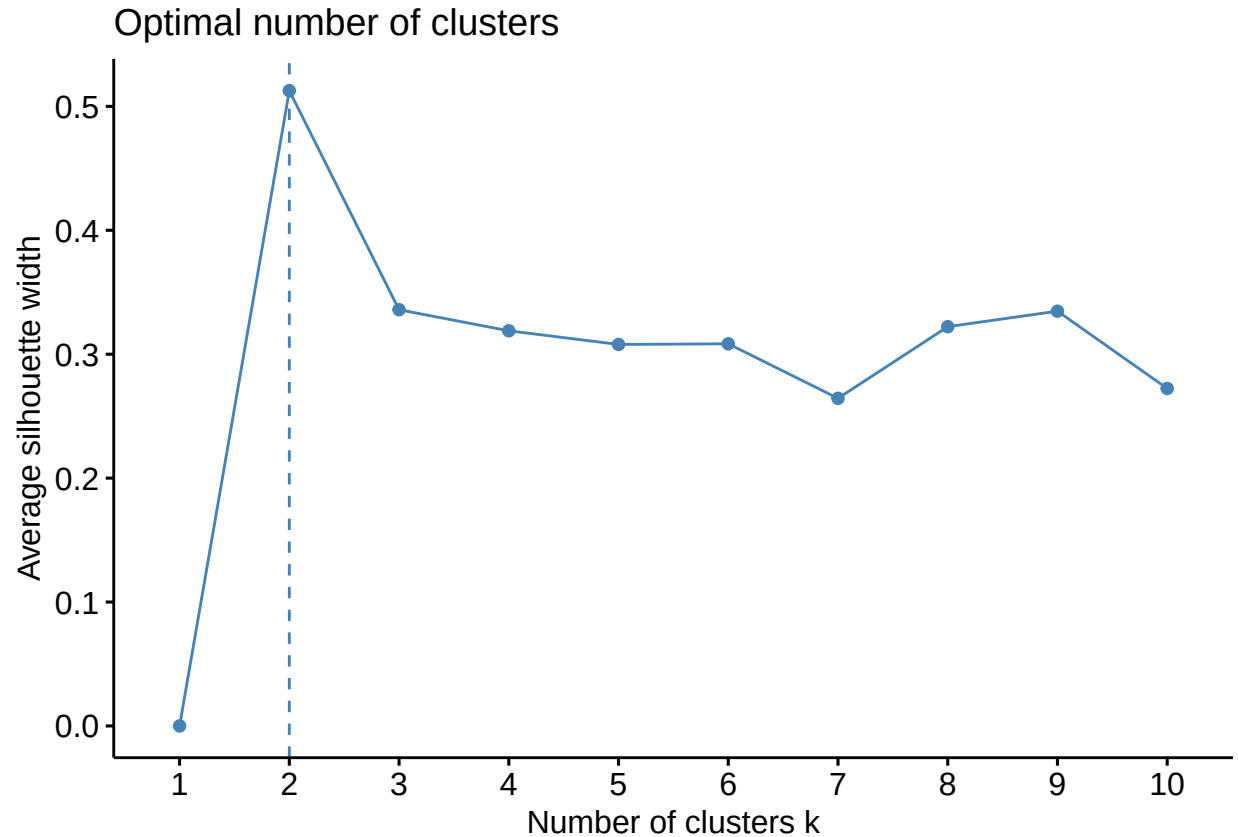
Choice of the Number of Clusters

```
set.seed(123)

fviz_nbclust(feat_mat, kmeans, method = "wss")
```



```
fviz_nbclust(feat_mat, kmeans, method = "silhouette")
```



We chose $k=2$ based on both the elbow and silhouette criteria. In the WSS (elbow) plot, the largest drop occurs between $k=1$ and $k=2/3$, and the curve flattens after $k=2$, indicating limited gain from additional clusters. The average silhouette width is highest at $k=2$, suggesting that two clusters provide the best separation and cohesion.

K-means Clustering Results

```
library(gt)

cluster_table <- cluster_summary |>
  mutate(
    cluster = if_else(cluster == 1, "Cluster 1", "Cluster 2")
  ) |>
  gt(rowname_col = "cluster") |>
  tab_header(
    title = md("**Climate characteristics by cluster**")
  ) |>
  cols_label(
    n_counties = "n",
    mean_tmmx_c = "Mean tmmx ( $\circ$ C)",
    mean_tmmn_c = "Mean tmmn ( $\circ$ C)",
    mean_rmax = "Mean rmax (%)",
    mean_vpd = "Mean VPD (kPa)",
    mean_pr = "Mean precip (mm)",
```

Climate characteristics by cluster

	n	Mean tmmx ($^{\circ}\text{C}$)	Mean tmmn ($^{\circ}\text{C}$)	Mean rmax (%)	Mean VPD (kPa)	Mean precip
Cluster 1	9	17.02	8.37	81.06	0.65	
Cluster 2	53	13.77	3.00	89.54	0.48	

```

    mean_srad = "Mean srad (W/m2)",
    mean_vs   = "Mean wind (m/s)"
  ) |>
  fmt_number(
    columns = c(mean_tmmx_c:mean_vs),
    decimals = 2
  ) |>
  tab_options(
    table.font.size = px(14),
    data_row.padding = px(4)
  )

cluster_table

```

We identified two clusters of New York counties based on their 2016–2020 average meteorological profiles.

Cluster 1 (n = 9) exhibits warmer and sunnier conditions, with higher mean daily maximum and minimum temperature (approximately 17.0°C and 8.4°C) compared with Cluster 2 (13.8°C and 3.0°C).

Cluster 1 also has lower maximum relative humidity and higher vapor pressure deficit, indicating relatively drier air, alongside slightly higher precipitation, stronger solar radiation, and higher wind speed.

Cluster 2 (n = 53), which contains most counties, is characterized by cooler temperatures, higher humidity, lower solar radiation, and slightly weaker winds.

Thus, Cluster 1 can be interpreted as a “warmer, sunnier, relatively drier” climate type, whereas Cluster 2 represents a “cooler, more humid” climate type.

Principal Component Analysis

From the perspective of these data, PCA is useful because there are many correlated climate features at the county level.

Considering all seven variables separately makes it difficult to see the overall structure of the data or to understand how the k-means clusters differ in the multivariate space.

PCA provides a set of uncorrelated components that summarize the main gradients of climate variation, allowing us to visualize the data in only two dimensions.

We focus on PC1 and PC2 because they explain the largest share of total variance and therefore capture the most important patterns in the climate features.

PC1 (horizontal axis) represents the primary gradient of climate variation. Counties in Cluster 1 and Cluster 2 fall on opposite sides, so PC1 separates warmer, sunnier, relatively drier counties from cooler, more humid counties.

PC2 (vertical axis) captures additional variation not explained by PC1, such as smaller differences in humidity, precipitation, radiation, and wind speed, and helps show within-cluster heterogeneity.

Visualization of Climate-Based Clusters

After choosing $k=2$, we used three complementary plots to evaluate whether the clusters are meaningful and how they differ:

1. How do the original climate variables differ by cluster?
2. Do the clusters separate well in the multivariate feature space?
3. How are the clusters distributed geographically across New York State?

```
plot_data <- county_cluster |>
select(county, cluster, mean_tmmx_c:mean_vs) |>
pivot_longer(
  cols = -c(county, cluster),
  names_to = "variable",
  values_to = "value"
)

ggplot(plot_data, aes(x = cluster, y = value, fill = cluster)) +
  geom_boxplot(alpha = 0.7) +
  facet_wrap(~ variable, scales = "free_y") +
  theme_bw() +
  labs(
    x = "Cluster",
    y = "Value",
    title = "Distribution of climate features by cluster"
  ) +
  theme(legend.position = "none")
```

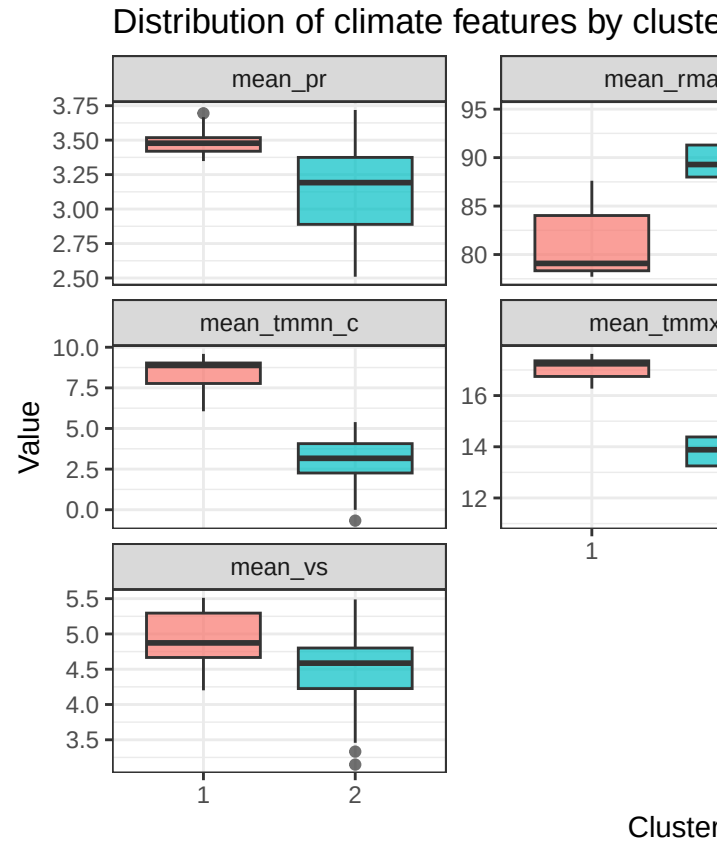


Figure 1: Distribution of Climate Features by Cluster

The faceted boxplots show the distribution of each county-level climate feature by cluster.

They highlight that Cluster 1 counties tend to have higher mean maximum and minimum temperatures, higher solar radiation and wind speed, slightly higher precipitation, lower maximum relative humidity, and higher vapor pressure deficit than Cluster 2, making the “warm, sunny, relatively dry” vs “cooler, more humid” contrast clear on the original scale of the variables.

```
pca_fit <- prcomp(feet_mat, center = FALSE, scale. = FALSE)

pca_df <- as.data.frame(pca_fit$x[, 1:2]) |>
mutate(
  county = county_feet$county,
  cluster = county_cluster$cluster
)

ggplot(pca_df, aes(x = PC1, y = PC2, color = cluster)) +
  geom_point(size = 3) +
  theme_minimal() +
  labs(
    title = "PCA of county-level climate features",
    x = "PC1",
    y = "PC2",
    color = "Cluster"
  )
```

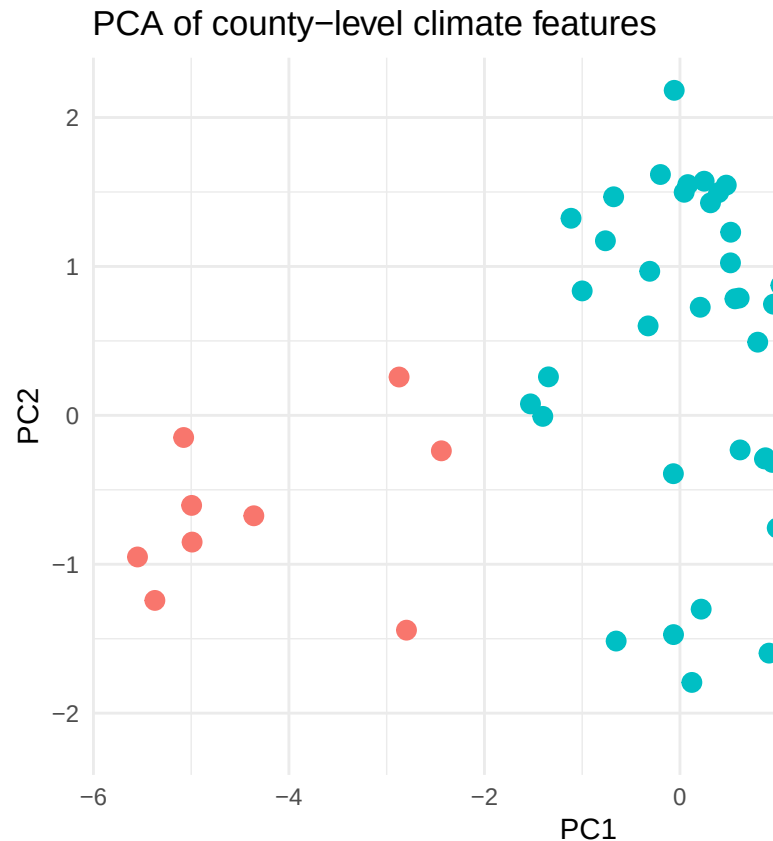



Figure 2: PCA of County-Level Climate Features

The PCA plot projects all standardized features onto the first two principal components and colors points by cluster.

Cluster 1 counties cluster together on one side of the PC1 axis, while Cluster 2 counties occupy a separate region, indicating that the two clusters are well separated in the reduced multivariate climate space.

```
county_cluster2 <- county_cluster |>
mutate(fips = as.character(county))

ny_sf2 <- county_fips_df |>
mutate(fips = as.character(fips))

ny_map <- ny_sf2 |>
left_join(county_cluster2, by = "fips")

ggplot(ny_map) +
  geom_sf(aes(fill = cluster), color = "white", size = 0.2) +
  scale_fill_brewer(palette = "Set2", name = "Cluster") +
  theme_minimal() +
  labs(
    title = "Climate-based clusters of New York counties (k = 2)"
  )
```

Climate-based clusters of New York counties

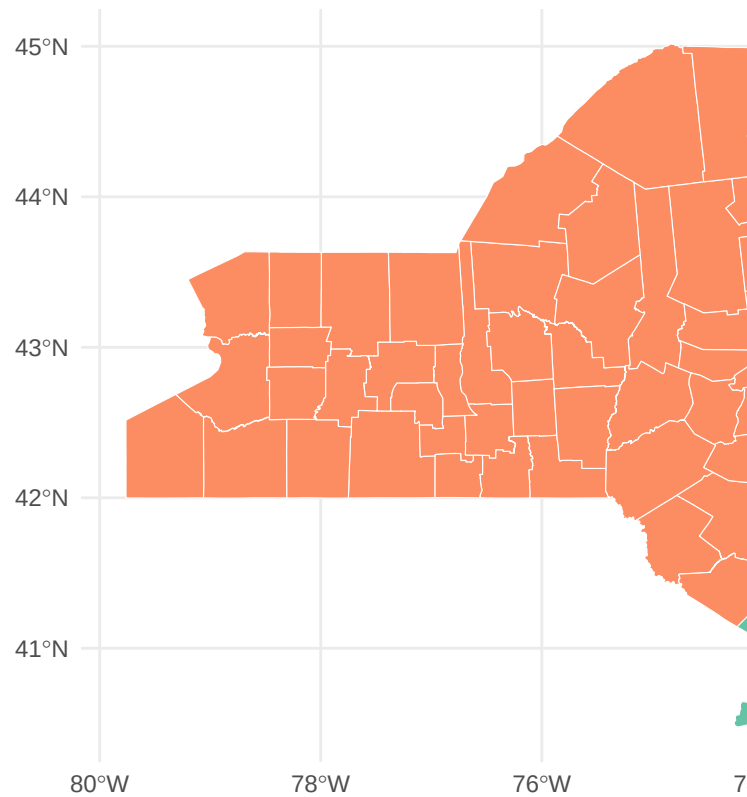


Figure 3: Spatial Distribution of Climate Clusters

The choropleth map joins cluster labels to the county shapefile and shows that Cluster 1 counties are concentrated in the downstate/NYC and Long Island region, whereas most upstate and interior counties belong to Cluster 2.

This confirms that the climate-based clusters also align with intuitive coastal vs inland geographic patterns.

Summary

Combining k-means clustering with PCA and spatial visualization demonstrates that New York counties can be grouped into two distinct climate types.

The first cluster represents warmer, sunnier, relatively drier coastal counties, while the second cluster captures cooler, more humid interior counties.

These patterns are consistent across the original climate features, the low-dimensional PCA representation, and the geographic distribution.

Prediction Models

```
library(readr)
library(dplyr)
library(lubridate)
library(forecast)
library(tidyr)
```

```
library(ggplot2)
library(tibble)
library(knitr)
library(forecast)
library(lubridate)
library(xgboost)
library(stats)
library(grid)
```

Model 1

Data preparation

In this section we aggregate station-level meteorological data to the New York State level and create lagged covariates that will be used as external regressors in the ARIMAX model.

```
data_raw <- read_csv("df_gen/data/meteor_2016to20_NY.csv")
dat <- data_raw |>
  mutate(
    tmmx = tmmx - 273.15,
    tmmn = tmmn - 273.15,
    year  = year(date),
    month = month(date)
  )
dat_state <- dat %>%
  group_by(date) %>%
  summarise(
    tmmx = mean(tmmx, na.rm = TRUE),
    sph  = mean(sph,   na.rm = TRUE),
    vpd  = mean(vpd,   na.rm = TRUE),
    srad = mean(srad,  na.rm = TRUE),
    vs   = mean(vs,    na.rm = TRUE)
  ) %>%
  arrange(date)

dat_state_lag <- dat_state %>%
  mutate(
    sph_lag1 = dplyr::lag(sph, 1),
    vpd_lag1 = dplyr::lag(vpd, 1),
    srad_lag1 = dplyr::lag(srad, 1),
    vs_lag1  = dplyr::lag(vs, 1)
  ) %>%
  tidyr::drop_na()

start_year <- year(min(dat_state_lag$date))
start_doy  <- yday(min(dat_state_lag$date))

tmmx_ts <- ts(
  dat_state_lag$tmmx,
  start      = c(start_year, start_doy),
  frequency = 365
)
```

ARIMAX + Fourier model

To represent strong annual seasonality, we use Fourier terms instead of a seasonal ARIMA structure. These Fourier harmonics, together with lagged meteorological covariates, enter the model as external regressors, while the error term follows a non-seasonal ARIMA process selected by `auto.arima()`.

```
K <- 3
fourier_terms <- fourier(tmmx_ts, K = K)
xreg <- cbind(
  fourier_terms,
  sph_lag1 = dat_state_lag$sph_lag1,
  vpd_lag1 = dat_state_lag$vpd_lag1,
  srad_lag1 = dat_state_lag$srad_lag1,
  vs_lag1 = dat_state_lag$vs_lag1
)

fit_arimax_fourier <- auto.arima(
  tmmx_ts,
  xreg = xreg,
  seasonal = FALSE
)

model_info <- tibble(
  Metric = c("Log-Likelihood", "Sigma^2", "AIC", "AICc", "BIC"),
  Value = c(
    as.numeric(logLik(fit_arimax_fourier)),
    fit_arimax_fourier$sigma2,
    fit_arimax_fourier$aic,
    fit_arimax_fourier$aicc,
    fit_arimax_fourier$bic
  )
)

kable(
  model_info,
  digits = 4,
  caption = "Overall fit statistics for the ARIMAX + Fourier model"
)
```

Table 1: Overall fit statistics for the ARIMAX + Fourier model

Metric	Value
Log-Likelihood	-4819.6440
Sigma ²	11.5696
AIC	9669.2880
AICc	9669.5532
BIC	9751.9363

In this specification, the ARIMA component captures short-term temporal dependence, while Fourier terms describe the smooth annual cycle. Lagged humidity, vapor-pressure deficit, solar radiation, and wind speed account for additional meteorological drivers of temperature.

Coefficient summary

We present the estimated coefficients and standard errors for the AR terms, meteorological covariates, and Fourier harmonics.

```
coef_est <- coef(fit_arimax_fourier)
se_est   <- sqrt(diag(vcov(fit_arimax_fourier)))

coef_tbl <- tibble(
  Term      = names(coef_est),
  Estimate  = as.numeric(coef_est),
  SE        = as.numeric(se_est)
)

kable(
  coef_tbl,
  digits = 4,
  caption = "Coefficient estimates for ARIMAX + Fourier model"
)
```

Table 2: Coefficient estimates for ARIMAX + Fourier model

Term	Estimate	SE
ar1	0.7726	0.0410
ar2	-0.2497	0.0375
ar3	0.1078	0.0241
intercept	16.4607	0.9516
S1-365	-4.5286	0.3896
C1-365	-12.3698	0.7226
S2-365	0.5384	0.3231
C2-365	-0.4380	0.3095
S3-365	0.5627	0.3020
C3-365	-0.3102	0.3016
sph_lag1	-47.3621	101.2496
vpd_lag1	0.1925	0.5844
srad_lag1	0.0015	0.0023
vs_lag1	-0.4976	0.0547

Model Diagnostics

We assess model adequacy by examining residuals. Ideally, residuals should behave like uncorrelated noise with no systematic temporal pattern.

Residuals from Regression with ARIMA(3,0,0) errors

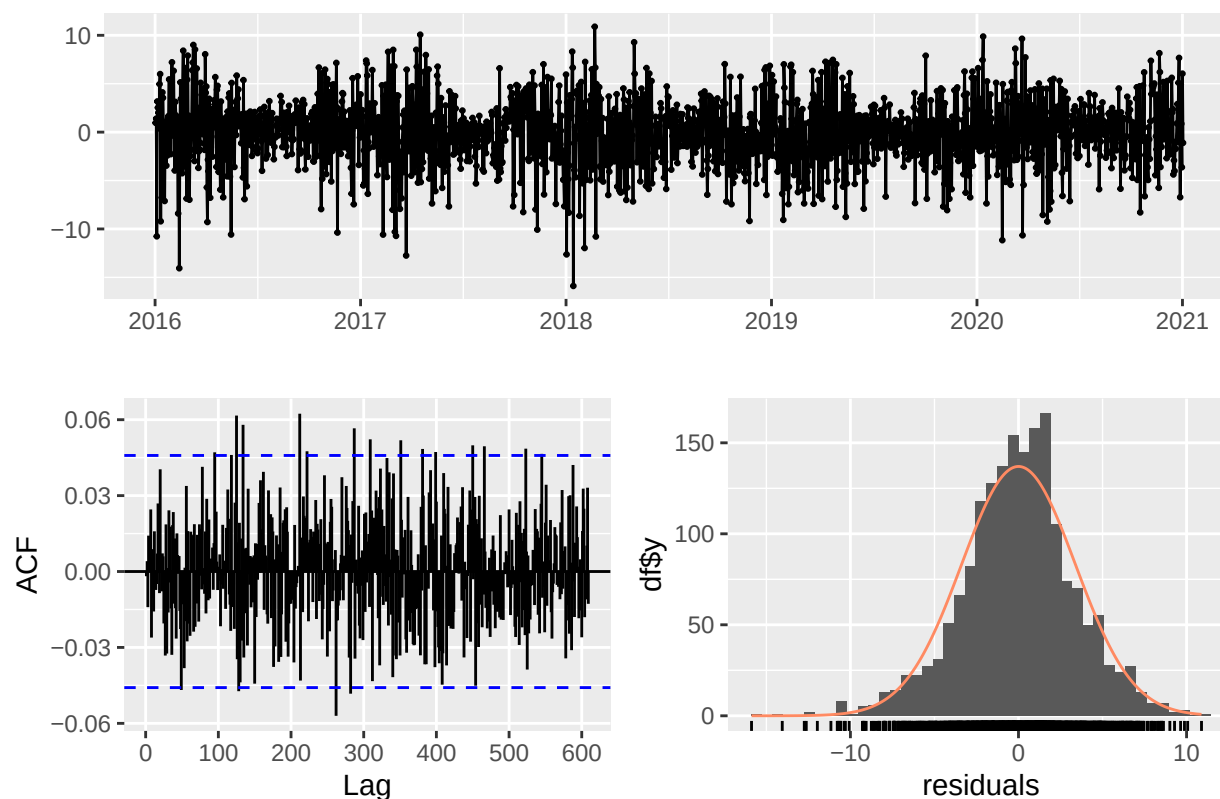


Table 3: Ljung–Box Test for ARIMAX + Fourier Residuals

Statistic	Degrees.of.freedom	P.value	Model.df	Total.lags.used
330.1	362	0.8844	3	365

The residual time series shows no obvious structure, the autocorrelation function remains within the 95% bounds at most lags, and the Ljung–Box test is non-significant. Together, these diagnostics suggest that the ARIMAX + Fourier model has captured most of the temporal dependence and seasonal pattern in the data, and the remaining residuals are approximately white noise.

Forecasting and Interpretation

We generate multi-step-ahead forecasts using the fitted ARIMAX + Fourier model. Future Fourier terms are constructed analytically, and recent lagged covariates are used to build the regressor matrix for the forecast horizon.

```
h <- 730
fourier_future <- fourier(tmmx_ts, K = K, h = h)

recent_block <- dat_state_lag %>%
  slice_tail(n = 365)

xreg_future <- cbind(
  fourier_future,
```

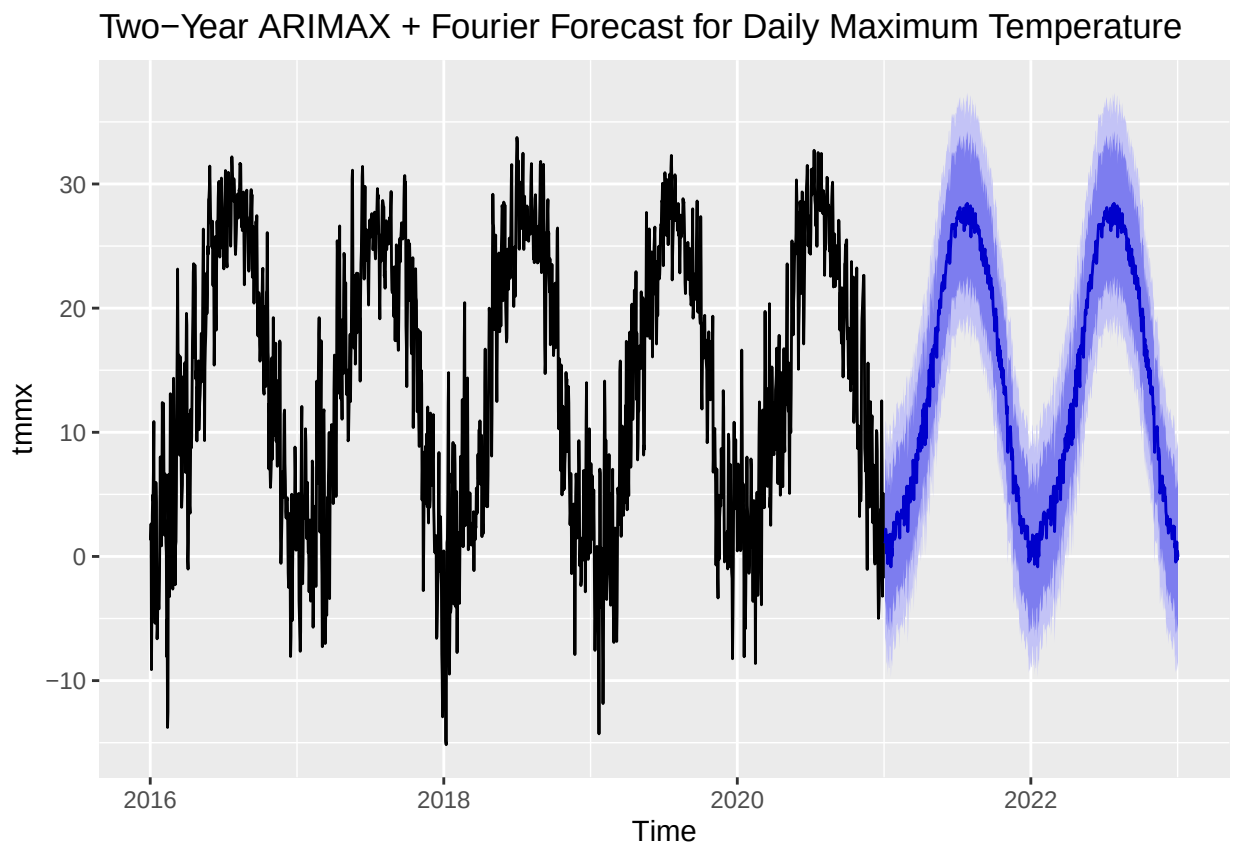
```

sph_lag1 = recent_block$sph_lag1,
vpd_lag1 = recent_block$vpd_lag1,
srad_lag1 = recent_block$srad_lag1,
vs_lag1 = recent_block$vs_lag1
)

fc <- forecast(fit_arimax_fourier, xreg = xreg_future, h = h)

autoplot(fc) +
ggtitle("Two-Year ARIMAX + Fourier Forecast for Daily Maximum Temperature") +
xlab("Time") +
ylab("tmmx")

```



The figure presents a two-year forecast of daily maximum temperature derived from an ARIMAX model augmented with Fourier seasonal terms. The historical portion of the time series (black) exhibits strong annual cycles and substantial short-term variability, both of which are captured by the model. The forecasted values (blue) follow a smooth, periodic pattern consistent with the dominant seasonal structure. The incorporation of Fourier harmonics allows the model to represent complex seasonal behavior more flexibly than classical seasonal ARIMA, resulting in coherent long-range forecasts that align with the historical climatological patterns.

Model 2

Data Preparation

Daily statewide averages for all available meteorological variables were computed. Feature engineering included lag terms, rolling means, and time-based indicators (day of year, weekday, year), which help XGBoost capture temporal dependencies and short-term memory effects.

```
dat <- dat %>%
mutate(date = as.Date(date))

dat_state <- dat %>%
group_by(date) %>%
summarise(
  tmmx = mean(tmmx, na.rm = TRUE),
  tmmn = mean(tmmn, na.rm = TRUE),
  sph = mean(sph, na.rm = TRUE),
  vpd = mean(vpd, na.rm = TRUE),
  pr = mean(pr, na.rm = TRUE),
  rmin = mean(rmin, na.rm = TRUE),
  rmax = mean(rmax, na.rm = TRUE),
  srاد = mean(srad, na.rm = TRUE),
  vs = mean(vs, na.rm = TRUE),
  th = mean(th, na.rm = TRUE)
) %>%
ungroup() %>%
arrange(date)

dat_ml <- dat_state %>%
mutate(
  doy = yday(date),
  dow = wday(date, week_start = 1),
  year = year(date),
  tmmx_lag1 = lag(tmmx, 1),
  tmmx_lag2 = lag(tmmx, 2),
  tmmx_lag7 = lag(tmmx, 7),
  tmmn_lag1 = lag(tmmn, 1),
  sph_lag1 = lag(sph, 1),
  vpd_lag1 = lag(vpd, 1),
  pr_lag1 = lag(pr, 1),
  rmin_lag1 = lag(rmin, 1),
  rmax_lag1 = lag(rmax, 1),
  srad_lag1 = lag(srad, 1),
  vs_lag1 = lag(vs, 1)
)

dat_ml$tmmx_roll17 <- as.numeric(stats::filter(dat_ml$tmmx, rep(1/7, 7), sides = 1))

dat_ml <- dat_ml %>% drop_na()
```


XGBoost Model

The dataset was split chronologically (80% training, 20% testing). An XGBoost regression model was trained using RMSE as the optimization metric and early stopping to prevent overfitting.

```
split_date <- dat_ml$date[floor(nrow(dat_ml) * 0.8)]
train_df   <- dat_ml %>% filter(date <= split_date)
test_df    <- dat_ml %>% filter(date > split_date)

feature_cols <- c(
  "tmmn", "sph", "vpd", "pr", "rmin", "rmax", "srad", "vs", "th",
  "tmmx_lag1", "tmmx_lag2", "tmmx_lag7",
  "tmmn_lag1", "sph_lag1", "vpd_lag1", "pr_lag1",
  "rmin_lag1", "rmax_lag1", "srad_lag1", "vs_lag1",
  "tmmx_roll7", "doy", "dow", "year"
)

train_x <- as.matrix(train_df[, feature_cols])
train_y <- train_df$tmmx

test_x <- as.matrix(test_df[, feature_cols])
test_y <- test_df$tmmx

dtrain <- xgb.DMatrix(train_x, label = train_y)
dtest  <- xgb.DMatrix(test_x, label = test_y)

params <- list(
  objective      = "reg:squarederror",
  eval_metric    = "rmse",
  max_depth     = 5,
  eta           = 0.05,
  subsample     = 0.8,
  colsample_bytree = 0.8
)

set.seed(2025)
xgb_model <- xgb.train(
  params      = params,
  data        = dtrain,
  nrounds     = 1500,
  evals       = list(train = dtrain, eval = dtest),
  early_stopping_rounds = 50,
  verbose     = 0
)

test_pred <- predict(xgb_model, dtest)

test_df <- test_df %>%
  mutate(
    pred = test_pred,
    resid = tmmx - pred
  )

rmse <- sqrt(mean((test_df$pred - test_df$tmmx)^2))
```

```

mae <- mean(abs(test_df$pred - test_df$tmmx))
mape <- mean(abs((test_df$pred - test_df$tmmx) / test_df$tmmx)) * 100
ss_res <- sum((test_df$tmmx - test_df$pred)^2)
ss_tot <- sum((test_df$tmmx - mean(test_df$tmmx))^2)
r2 <- 1 - ss_res / ss_tot

perf_xgb <- tibble::tibble(
  Metric = c("RMSE", "MAE", "MAPE (%)", "R2"),
  Value = c(rmse, mae, mape, r2)
)

knitr::kable(perf_xgb, digits=3, caption="XGBoost Model Performance (Test Set)")

```

Table 4: XGBoost Model Performance (Test Set)

Metric	Value
RMSE	0.574
MAE	0.435
MAPE (%)	112.168
R ²	0.997

Model Diagnostics

Residual analysis evaluates whether the model captures structure adequately and whether remaining errors behave randomly.

```

p2 <- ggplot(test_df, aes(x = date)) +
  geom_line(aes(y = tmmx, color = "Observed"), linewidth = 0.7) +
  geom_line(aes(y = pred, color = "Predicted"), linewidth = 0.7) +
  scale_color_manual(values = c("Observed" = "black", "Predicted" = "blue")) +
  labs(title = "Observed vs Predicted (Test Set)",
       x = "Date", y = "tmmx") +
  theme_minimal() +
  theme(legend.title = element_blank(),
        legend.position = "bottom")

p3 <- ggplot(test_df, aes(x = date, y = resid)) +
  geom_line() +
  labs(title = "Residuals Over Time",
       x = "Date", y = "Residual") +
  theme_minimal()

acf_obj <- acf(test_df$resid, plot = FALSE)
acf_df <- with(acf_obj, data.frame(lag = lag, acf = acf))
n <- length(test_df$resid)
ci <- 1.96 / sqrt(n)

p5 <- ggplot(acf_df, aes(x = lag, y = acf)) +
  geom_hline(yintercept = 0) +
  geom_segment(aes(xend = lag, yend = 0)) +
  geom_hline(yintercept = c(-ci, ci),
            linetype = "dashed", colour = "blue") +

```

```

labs(title = "Residual ACF", x = "Lag", y = "ACF") +
theme_minimal()

p4 <- ggplot(test_df, aes(x = tmmx, y = pred)) +
  geom_point(alpha = 0.4) +
  geom_abline(slope = 1, intercept = 0,
              colour = "red", linetype = "dashed") +
  labs(title = "Predicted vs Observed (Test Set)",
        x = "Observed tmmx", y = "Predicted tmmx") +
  theme_minimal()
p1 <- ggplot(test_df, aes(x = resid)) +
  geom_histogram(bins = 30, colour = "white", fill = "steelblue") +
  labs(title = "Residual Distribution (Test Set)",
        x = "Residual", y = "Count") +
  theme_minimal()

multiplot <- function(..., plotlist=NULL, cols=1) {
  library(grid)

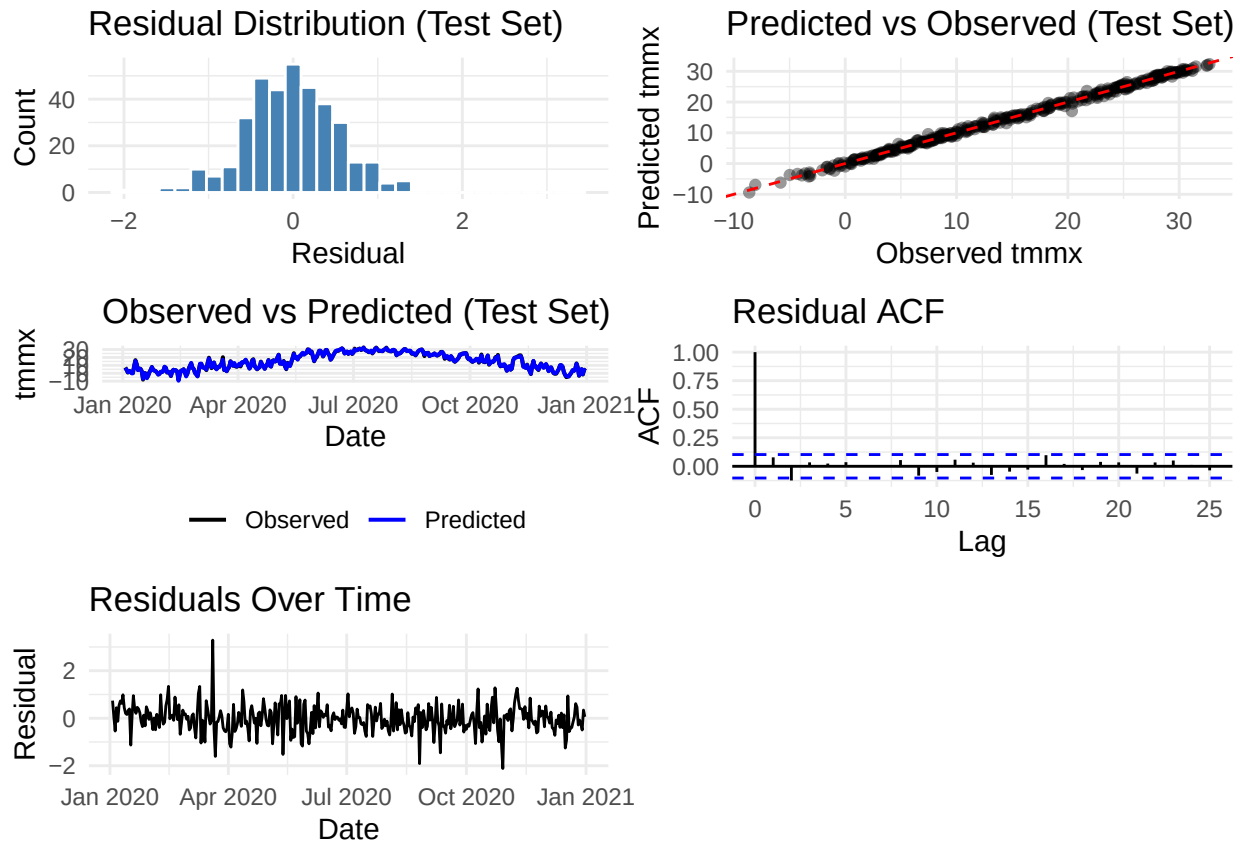
  plots <- c(list(...), plotlist)
  numPlots <- length(plots)

  layout <- matrix(seq(1, cols * ceiling(numPlots / cols)),
                    ncol = cols, nrow = ceiling(numPlots / cols))

  grid.newpage()
  pushViewport(viewport(layout = grid.layout(nrow(layout), ncol(layout))))
  for (i in 1:numPlots) {
    matchidx <- as.data.frame(which(layout == i, arr.ind = TRUE))
    print(plots[[i]], vp = viewport(
      layout.pos.row = matchidx$row,
      layout.pos.col = matchidx$col
    ))
  }
}

multiplot(p1,p2, p3, p4, p5, cols = 2)

```



The XGBoost model demonstrates strong predictive performance, with predicted values closely tracking observed temperatures across the test period. Residuals are centered around zero with no visible temporal patterns, and the ACF shows no meaningful autocorrelation, indicating well-behaved errors. The predicted-versus-observed scatterplot aligns tightly around the 45-degree line, and the residual distribution is approximately normal with small variance. Overall, the diagnostics suggest that XGBoost effectively captures the nonlinear and seasonal structure of the temperature series and produces accurate, unbiased predictions.

Forecasting and Interpretation

Future meteorological covariates were constructed by cycling the most recent 365-day pattern, enabling the model to simulate realistic seasonal dynamics.

```
dat_state <- dat %>%
  mutate(date = as.Date(date)) %>%
  group_by(date) %>%
  summarise(
    tmmx = mean(tmmx, na.rm = TRUE),
    tmmn = mean(tmmn, na.rm = TRUE),
    sph = mean(sph, na.rm = TRUE),
    vpd = mean(vpd, na.rm = TRUE),
    pr = mean(pr, na.rm = TRUE),
    rmin = mean(rmin, na.rm = TRUE),
    rmax = mean(rmax, na.rm = TRUE),
    srads = mean(srads, na.rm = TRUE),
```

```

    vs = mean(vs, na.rm = TRUE),
    th = mean(th, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  arrange(date)

last_365_block <- dat_state %>%
  slice_tail(n = 365) %>%
  select(tmmn, sph, vpd, pr, rmin, rmax, srad, vs, th)

h <- 730
future_list <- vector("list", length = h)

state <- dat_ml %>%
  arrange(date) %>%
  slice_tail(n = 7)

last_date <- max(state$date)

for (i in seq_len(h)) {

  new_date <- last_date + i
  idx <- ((i - 1) %% 365) + 1
  exo_today <- last_365_block[idx, ]

  tmp <- tibble::tibble(
    date = new_date,
    tmmn = exo_today$tmmn,
    sph = exo_today$sph,
    vpd = exo_today$vpd,
    pr = exo_today$pr,
    rmin = exo_today$rmin,
    rmax = exo_today$rmax,
    srad = exo_today$srad,
    vs = exo_today$vs,
    th = exo_today$th,
    doy = yday(new_date),
    dow = wday(new_date, week_start = 1),
    year = year(new_date)
  )

  tmp <- tmp %>%
    mutate(
      tmmx_lag1 = state$tmmx[nrow(state)],
      tmmx_lag2 = state$tmmx[nrow(state) - 1],
      tmmx_lag7 = state$tmmx[nrow(state) - 6],

      tmmn_lag1 = state$tmmn[nrow(state)],
      sph_lag1 = state$sph[nrow(state)],
      vpd_lag1 = state$vpd[nrow(state)],

```

```

pr_lag1      = state$pr[nrow(state)],
rmin_lag1    = state$rmin[nrow(state)],
rmax_lag1    = state$rmax[nrow(state)],
srاد_lag1    = state$srاد[nrow(state)],
vs_lag1      = state$vs[nrow(state)],

tmmx_roll7 = mean(state$tmmx)
)

x_new <- as.matrix(tmp[, feature_cols])

pred_new <- predict(xgb_model, xgb.DMatrix(x_new))

tmp$tmmx <- as.numeric(pred_new)

future_list[[i]] <- tmp

state <- dplyr::bind_rows(state, tmp)
state <- state %>% slice_tail(n = 7)
}

future_730 <- dplyr::bind_rows(future_list)

sd_resid <- sd(test_df$resid)

future_730 <- future_730 %>%
  mutate(
    tmmx_pred = tmmx,
    lower_95  = tmmx_pred - 1.96 * sd_resid,
    upper_95  = tmmx_pred + 1.96 * sd_resid
  )

past_all <- dat_state %>%
  arrange(date) %>%
  select(date, tmmx) %>%
  mutate(source = "Observed (all history)")

future_plot <- future_730 %>%
  select(date, tmmx_pred, lower_95, upper_95) %>%
  mutate(source = "XGBoost forecast")

ggplot() +
  geom_line(data = past_all,
            aes(x = date, y = tmmx, color = "Observed"),
            linewidth = 0.7) +

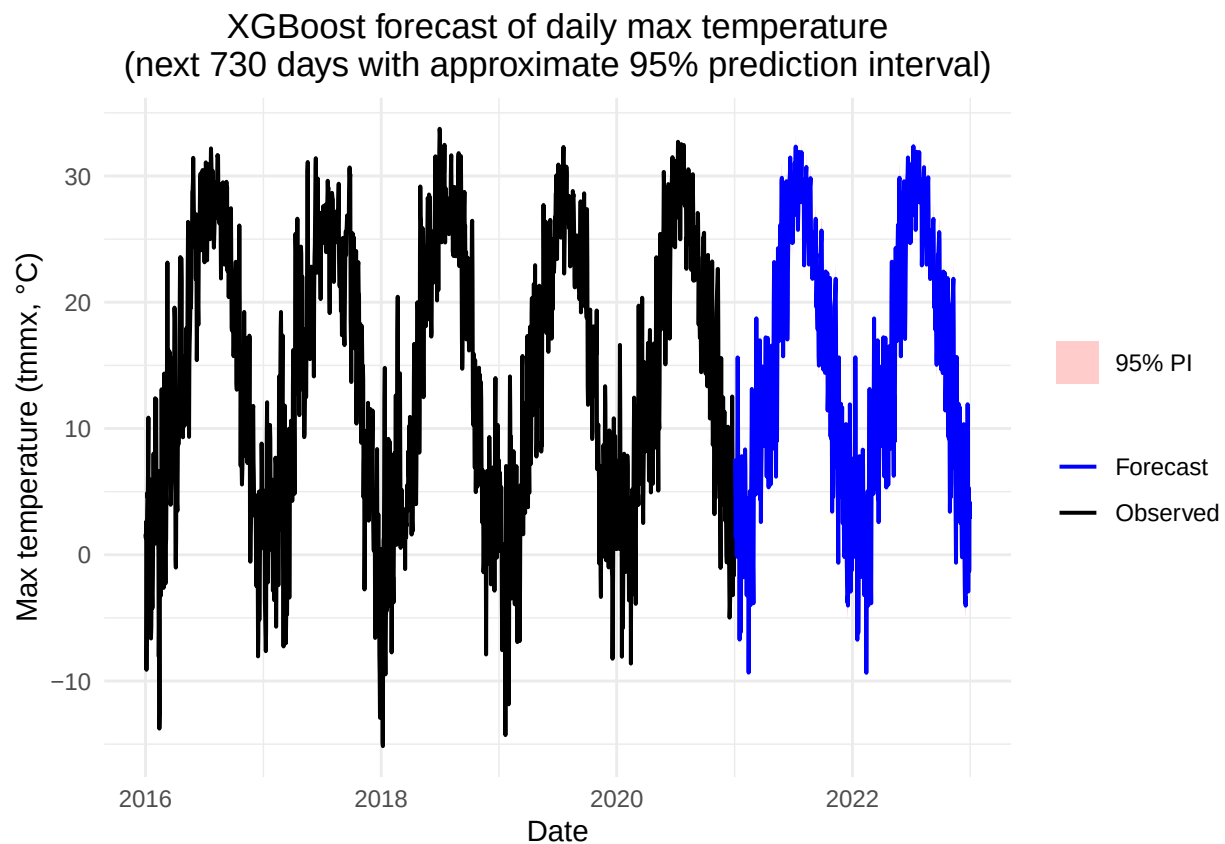
  geom_ribbon(data = future_plot,
             aes(x = date, ymin = lower_95, ymax = upper_95, fill = "95% PI"),
             alpha = 0.2, inherit.aes = FALSE) +
  geom_line(data = future_plot,
            aes(x = date, y = tmmx_pred, color = "Forecast"),

```

```

        linewidth = 0.7) +
labs(
  title = "XGBoost forecast of daily max temperature\n(next 730 days with approximate 95% prediction interval)",
  x = "Date",
  y = "Max temperature (tmmx, °C)"
) +
scale_color_manual(
  values = c("Observed" = "black", "Forecast" = "blue")
) +
scale_fill_manual(
  values = c("95% PI" = "red")
) +
theme_minimal() +
theme(
  legend.title = element_blank(),
  plot.title = element_text(hjust = 0.5)
)

```



This XGBoost model effectively captures nonlinear meteorological and temporal dependencies, delivering strong short-term predictive accuracy and stable long-horizon forecasts. The model serves as a complementary alternative to ARIMAX, offering improved flexibility in handling lagged covariates and complex interactions.

Model Comparison

```
ar_train <- dat_state_lag %>% filter(date <= split_date)
ar_test  <- dat_state_lag %>% filter(date >  split_date)

if (nrow(ar_test) == 0 | nrow(ar_train) == 0) {
  stop("ARIMAX train/test split produced empty set. Check date ranges of dat_state_lag vs dat_ml.")
}

start_year <- year(min(ar_train$date))
start_doy  <- yday(min(ar_train$date))

tmmx_ts_train <- ts(
  ar_train$tmmx,
  start      = c(start_year, start_doy),
  frequency = 365
)

K <- 3

fourier_train <- fourier(tmmx_ts_train, K = K)

xreg_train <- cbind(
  fourier_train,
  sph_lag1 = ar_train$sph_lag1,
  vpd_lag1 = ar_train$vpd_lag1,
  srad_lag1 = ar_train$srad_lag1,
  vs_lag1  = ar_train$vs_lag1
)

fit_arimax_cv <- auto.arima(
  tmmx_ts_train,
  xreg      = xreg_train,
  seasonal = FALSE
)

h_test <- nrow(ar_test)

fourier_test <- fourier(tmmx_ts_train, K = K, h = h_test)

xreg_test <- cbind(
  fourier_test,
  sph_lag1 = ar_test$sph_lag1,
  vpd_lag1 = ar_test$vpd_lag1,
  srad_lag1 = ar_test$srad_lag1,
  vs_lag1  = ar_test$vs_lag1
)

fc_test <- forecast(
  fit_arimax_cv,
  xreg = xreg_test,
  h    = h_test
)
```



```

)

ar_test <- ar_test %>%
  mutate(arimax_pred = as.numeric(fc_test$mean))

compare_df <- test_df %>%
  select(date, tmmx, xgb_pred = pred) %>%
  inner_join(ar_test %>% select(date, arimax_pred),
            by = "date") %>%
  arrange(date)

if (nrow(compare_df) == 0) {
  stop("No overlapping dates between XGBoost test_df and ARIMAX test set. Check date alignment.")
}

calc_metrics <- function(obs, pred) {
  rmse <- sqrt(mean((pred - obs)^2))
  mae <- mean(abs(pred - obs))
  mape <- mean(abs((pred - obs) / obs)) * 100
  ss_res <- sum((obs - pred)^2)
  ss_tot <- sum((obs - mean(obs))^2)
  r2 <- 1 - ss_res / ss_tot
  c(RMSE = rmse, MAE = mae, MAPE = mape, R2 = r2)
}

m_arimax <- calc_metrics(compare_df$tmmx, compare_df$arimax_pred)
m_xgb <- calc_metrics(compare_df$tmmx, compare_df$xgb_pred)

model_compare <- tibble::tibble(
  Model = c("ARIMAX + Fourier", "XGBoost"),
  RMSE = c(m_arimax["RMSE"], m_xgb["RMSE"]),
  MAE = c(m_arimax["MAE"], m_xgb["MAE"]),
  MAPE = c(m_arimax["MAPE"], m_xgb["MAPE"]),
  R2 = c(m_arimax["R2"], m_xgb["R2"])
)

kable(model_compare, digits = 3,
      caption = "Performance comparison of ARIMAX + Fourier vs XGBoost on the same test dates")

```

Table 5: Performance comparison of ARIMAX + Fourier vs XGBoost on the same test dates

Model	RMSE	MAE	MAPE	R2
ARIMAX + Fourier	8.593	7.608	1151.313	0.291
XGBoost	0.574	0.435	112.168	0.997

The test-set results show that XGBoost substantially outperforms the ARIMAX + Fourier model across all metrics, with far lower RMSE (0.57 vs. 8.59), MAE (0.43 vs. 7.61), and MAPE (112% vs. 1151%), as well as a much higher R^2 (0.997 vs. 0.291). These differences are consistent with the theoretical design of the two models. The ARIMAX + Fourier approach relies on linear relationships, predefined seasonal components, and a limited set of lagged regressors; as a result, it struggles to capture nonlinear temperature dynamics and complex interactions among meteorological variables. In contrast, XGBoost is a gradient-boosted ensemble model capable of learning highly flexible, nonlinear relationships and automatically capturing interactions

among features, including multiple lags, rolling averages, and time-of-year effects. This flexibility allows it to model temperature patterns far more accurately. However, ARIMAX has advantages in interpretability and classical time-series structure, while XGBoost, despite its higher accuracy, behaves more like a black box and requires more computational resources. Overall, the comparison suggests that XGBoost is markedly better suited for predictive performance in this dataset, especially when nonlinear and interaction effects are important.

Discussion

Discussion

This unsupervised analysis shows that New York counties naturally group into two interpretable climate types based on long-term averages of temperature, humidity, precipitation, solar radiation, and wind speed. One cluster corresponds to warmer, sunnier and relatively drier counties, while the other represents cooler, more humid counties with lower solar radiation. The same pattern appears in the summaries, the PCA plot, and the map, and matches the expected coastal–inland contrast (downstate/NYC and Long Island vs upstate interior), suggesting that the clusters capture a real and useful climate gradient rather than a purely algorithmic artifact.

The forecasting analysis evaluates two contrasting modeling strategies, ARIMAX with Fourier terms and XGBoost, using statewide aggregated meteorological time series. Because the forecasting models are trained on the overall New York state average rather than cluster-specific series, they do not incorporate spatial heterogeneity identified in the clustering analysis. The ARIMAX model explicitly captures seasonality and is highly interpretable, but it tends to smooth short-term variability and struggles with nonlinear interactions among predictors. In contrast, XGBoost is better able to learn nonlinear and lagged relationships, producing higher predictive accuracy and more adaptive short-term dynamics, though at the expense of interpretability.

Limitations

This work has several limitations. Collapsing 2016–2020 daily data into simple long-term means makes the clusters easy to interpret but discards seasonality, year-to-year variability, and extreme events. The choice of $k = 2$, while supported by diagnostics, is only one possible resolution and yields unbalanced cluster sizes. Furthermore, the clustering is purely unsupervised and does not incorporate socioeconomic or health outcomes, so the resulting groups should be interpreted as descriptive climate patterns rather than causal risk categories.

The forecasting models also have notable limitations. First, because both ARIMAX and XGBoost were trained on statewide averages, they ignore the meaningful spatial differences revealed by the clustering; temperature dynamics may differ substantially across regions, and a single aggregated model cannot capture these local variations. ARIMAX may leave residual autocorrelation and fails to address nonlinear temperature–meteorology interactions, while XGBoost relies heavily on feature engineering and accumulates error during multi-step forecasting. Neither approach explicitly incorporates long-term climate trends or external drivers. Future extensions could include cluster-specific or spatially varying models, hybrid approaches, or the integration of broader climate indicators to improve robustness and capture regional heterogeneity.

Reference

Barnes, J., Sheffield, P., Graber, N., Jessel, S., Lanza, K., Limaye, V. S., Morrow, F., Sauthoff, A., Schmeltz, M. and Smith, S. (2024) ‘Chapter 7: Human health and safety’, in Stevens, A. (ed.) New York State Climate

Impacts Assessment [Interim version for public release]. Albany, NY: New York State Energy Research and Development Authority.

Bazrafshan, O., Zamani, H., Farokhzadeh, B. and Caloiero, T. (2025) ‘A spectral analysis-driven SARIMAX framework with Fourier terms for monthly dust concentration forecasting’, *Earth*, 6(4), 123.

Hyndman, R. J. and Athanasopoulos, G. (2018) *Forecasting: Principles and Practice*, 2nd edn. Melbourne: OTexts. Available at: <https://otexts.com/fpp2/> (Accessed 6 December 2025).

Lamie, C., Bader, D., Graziano, K., Horton, R., John, K., O’Hern, N. and Spungin, S. (2024) ‘Chapter 2: New York State’s changing climate’, in *New York State Climate Impacts Assessment* [Interim version for public release]. Albany, NY: New York State Energy Research and Development Authority.

Peel, M. C., Finlayson, B. L. and McMahon, T. A. (2007) ‘Updated world map of the Köppen–Geiger climate classification’, *Hydrology and Earth System Sciences*, 11(5), pp. 1633–1644.